# VESM v1.1

Verified Engineering System Methodology
Consolidated methodology update based on the latest engineering, integration, adversarial-verification, release, and scaling learnings.

Version: 1.1 Status: Active methodology Updated: 17 August 2026

**Core principle: Build from evidence, freeze the mission, verify every boundary, degrade safely, minimize change surface, and scale only when measured reality justifies it.**

## 1. Purpose and Scope

VESM is the user's combined engineering methodology for building reliable software under strict mission, evidence, verification, quality, simplicity, and release constraints. It combines the previously established VVBM, VQS, GS 99.999, Git Simplicity & Lowest Risk Rule, and the later verification and operational learnings into one working system.

### VESM governs:

- Mission definition and scope control.
- Architecture before implementation.
- Evidence-first engineering decisions.
- One verified actionable step at a time during execution.
- Immutable domain modeling before business logic.
- Verification and adversarial testing.
- Quality and release readiness.
- Minimal-risk correction of frozen systems.
- Operational readiness and future scaling.

## 2. VESM Foundation

- VVBM — Vision Verified Build Mode: evidence-first decisions, architecture before implementation, no assumptions, targeted fixes, phase-by-phase verification, frozen mission, adversarial review, and release discipline.
- VQS — Verification Quality Standard: defines quality maturity from personal projects through mission-critical systems, with correctness, reliability, performance, security, and maintainability as standing dimensions.
- GS 99.999: mission-first, safety/reliability/evidence prioritized, and architectural decisions justified by measurable benefit where high assurance is required.
- Git Simplicity & Lowest Risk Rule: choose the simplest verified Git workflow and keep source-control problems separate from software-engineering problems.
- VESM: the combined operating methodology that applies these principles continuously rather than as isolated checklists.

## 3. Frozen Mission and Scope Discipline

- Freeze the mission before implementation after sufficient market/problem evidence where applicable.

- Do not introduce feature creep near release.
- A frozen version is immutable by default.
- Post-freeze changes must be classified as either a contract-preserving defect correction or new-version/change-scope work.
- Release-phase decisions must preserve the frozen mission rather than optimize for novelty.
- When a project is complete, simplicity is a feature: do not overengineer the final release.

# 4. Immutable Domain Model Rule

Standing rule: Every educational or business-critical object must become an immutable domain model before business logic operates on it. Core entities such as Question, Reaction, Experiment, Element, GeneratedPaper, AssessmentResult, MarketSnapshot, PressureResult, and InferenceResult should have explicit domain semantics before calculators or orchestration layers consume them.

This reduces accidental mutation, clarifies contracts, and makes calculations easier to verify.

# 5. Evidence-First Engineering

Evidence-to-Claim Traceability Law — NEW in V1.1: Every PASS, FAIL, freeze, release, and readiness claim must trace to concrete evidence.

- Executable test output.
- Reproducible observation.
- Generated artifact.
- Authoritative external specification/source.
- Recorded operational evidence.

Reported evidence and independently observed evidence must not be treated as identical. A verification report is stronger when the underlying artifact or execution output is available.

# 6. Architecture and Contract Boundaries

Contract Boundary Verification Law — NEW in V1.1: Verify both sides of every important module boundary, not merely each module in isolation.

- Provider → normalizer → domain model.
- Domain model → persistence.
- Persistence → collector/timeframe retrieval.
- Calculator → calculator.
- Calculator → inference engine.
- Inference engine → renderer.
- External API → application startup/runtime failure handling.

Boundary verification must test semantic assumptions as well as types and method signatures. The Step 11 issues involving an invalid internal pressure attribute and incorrect Day start retrieval are examples of defects that boundary verification is designed to catch.

# 7. Failure-Path Symmetry

Failure-Path Symmetry Law — NEW in V1.1: For every important success path, explicitly ask what happens when the same boundary fails.

- Startup failure.
- Runtime failure.
- External-provider failure.
- Persistence failure.
- Calculation/evidence failure.
- Shutdown/restart failure.

Required behavior: fail explicitly, degrade safely, preserve nullable/unknown semantics, and never manufacture data or inference.

# 8. Verification Workflow

- Step 1 — Reproduce: demonstrate the actual issue or requirement.
- Step 2 — Isolate: identify the smallest contract or component responsible.
- Step 3 — Hypothesize: rank likely causes rather than guessing.
- Step 4 — Minimal correction: change the smallest verified surface.
- Step 5 — Regression verification: rerun the affected and relevant broader tests.
- Step 6 — Invariant check: explicitly confirm mathematics, architecture, mission, and scope remain unchanged.
- Step 7 — Freeze: freeze the corrected phase only after evidence supports the claim.

# 9. Change-Surface Minimization

Change-Surface Minimization Law — NEW in V1.1: After a defect is found, prefer the smallest change that closes the verified contract violation.

### Every corrective change should answer:

- What exact defect does it close?
- Why is this the smallest sufficient correction?
- What tests prove the defect is closed?
- What frozen behavior remains unchanged?
- Does the change belong in the current version or require a new version?

# 10. Freeze Integrity

Freeze Integrity Law — NEW in V1.1: A frozen phase is immutable by default. A post-freeze change requires explicit classification.

| Change type | Handling |
|---|---|
| Contract-preserving defect fix | Allowed; minimal correction + targeted regression verification |
| New behavior / feature | Not allowed in frozen version; schedule for a new version |
| Architecture expansion | New version/scope unless explicitly re-baselined |
| Performance optimization | Allowed only when measured and non-regressive |

# 11. Adversarial Verification

Adversarial verification is not a final checkbox. It is an attempt to break the frozen system while preserving the frozen contract.

- Missing/null fields.
- Partial data.
- Insufficient historical data.
- Malformed external responses.
- Authentication failures.
- Rate limiting and network timeouts.
- Stale data.
- Boundary timestamps.
- Duplicate records.
- Zero/empty values.
- Contradictory evidence.
- Provider/source isolation.
- Startup, runtime, and shutdown failures.

Safety property: missing evidence should become explicit insufficiency/unknown behavior rather than being silently converted into a meaningful-looking value.

# 12. Operational Reality Gate

Operational Reality Gate — NEW in V1.1: Do not collapse these into one claim:

> **Mathematical correctness ≠ adversarial correctness ≠ operational readiness.**

| Dimension | Question |
|---|---|
| Mathematical | Does the implementation correctly follow the frozen equations/rules? |
| Implementation | Does the code satisfy the frozen architecture and contracts? |
| Adversarial | Does it degrade safely under adverse conditions? |
| Operational | Does it behave reliably with real dependencies, workload, and environment? |
| Release | Is the artifact complete, reproducible, documented, and suitable for its target maturity level? |

# 13. VQS Quality Maturity

| Level | Target | Minimum quality emphasis |
|---|---|---|
| 1 | Personal projects | Correctness, basic tests, maintainability |
| 2 | Client projects | Reliability, integration, documentation, security baseline |

| | | |
|---|---|---|
| 3 | Public/Open Source | Regression, compatibility, security, reproducible release, operational evidence |
| 4 | Mission-Critical | High assurance, failure injection, deep traceability, extreme reliability, operational controls |

Verification Depth Scaling Law — NEW in V1.1: Verification rigor scales with system maturity and risk, not merely project size. Scale Mode increases the verification burden along with capacity.

# 14. Performance and Optimization

Performance-Without-Regression Rule — NEW in V1.1: A 0.5–1% measurable improvement is a valid engineering improvement if it is real, repeatable, and does not weaken correctness, reliability, security, maintainability, simplicity, or the frozen mission.

> **VESM Optimization Principle: Never optimize for the largest improvement. Optimize for the highest verified improvement per unit of engineering risk.**

### Required optimization loop:

```
Measured baseline → targeted optimization → measured result → regression verification → accept/reject.
```

# 15. Scale Mode Boundary

Scale Mode should begin with V2, not V1. V1 is the verified reference implementation. Scale Mode is justified only after real workload or operational evidence identifies a measurable constraint.

### Recommended progression:

```
V1.0.0 → VESM verification → operational observation → measurable scaling constraint → V2 problem statement freeze → Scale Mode → V2 implementation → adversarial verification → V2 release.
```

Scaling must be driven by measured constraints, not by speculative architecture or feature accumulation.

# 16. Release-Phase Discipline

- Freeze scope before release execution.
- Do not fantasize or speculate about future features during release.
- Perform automated verification before declaring release.
- Use independent/adversarial review where practical.
- Prepare README, license, .gitignore, release notes, and reproducible setup according to project maturity.
- A release candidate is not complete merely because the code runs; the artifact and verification evidence must also be complete.

# 17. One Verified Step Execution Rule

During implementation and release execution, provide exactly one actionable step unless the user explicitly requests an overall plan. After each step: verify the result, acknowledge success/failure, and provide only the next step.

This rule exists to reduce command/path trial-and-error and prevent unverified branching during hands-on engineering.

# 18. VESM Decision Hierarchy

When principles conflict, prefer:

- Frozen mission over feature expansion.
- Evidence over assumption.
- Correctness over performance.
- Safety/reliability over convenience.
- Smallest verified change over broad refactoring.
- Simplicity over unnecessary abstraction.
- Measured optimization over intuition.
- Explicit insufficiency over fabricated certainty.
- Reproducibility over anecdotal success.
- Verified operational evidence over theoretical readiness.

# 19. VESM 1.1 Master Checklist

- 1. Mission defined and frozen.
- 2. Architecture defined before implementation.
- 3. Core domain objects immutable.
- 4. Business logic separated from data providers/persistence/UI.
- 5. Every important module boundary contract verified.
- 6. Happy paths and equivalent failure paths identified.
- 7. Missing evidence remains missing/unknown rather than fabricated.
- 8. Automated tests exist for the frozen contract.
- 9. Adversarial tests cover adverse inputs and external failures.
- 10. Verification claims trace to concrete evidence.
- 11. Post-freeze changes classified explicitly.
- 12. Corrective changes use minimum change surface.
- 13. Mathematics, architecture, and scope re-verified after fixes.
- 14. Operational readiness assessed separately from mathematical/adversarial correctness.
- 15. Performance changes benchmarked against a baseline.
- 16. Even small measurable improvements may be accepted when risk-free.
- 17. Scale Mode begins only when measured constraints justify it.
- 18. Release artifact is reproducible and complete.
- 19. No feature creep during final release.

# 20. VESM v1.1 — Formal Statement

VESM v1.1: A mission-first, evidence-driven, contract-aware engineering methodology that builds in small verified phases; models immutable domain state before business logic; verifies both success and failure paths; preserves explicit uncertainty; minimizes corrective change surfaces; protects frozen contracts; scales verification with maturity and risk; and permits only measured, non-regressive optimization.

Core invariant: No improvement is valuable enough to justify silently weakening correctness, reliability, security, maintainability, simplicity, evidence quality, or the frozen mission.

**Version boundary: V1 is the verified reference. V2 is the natural entry point for Scale Mode when operational evidence demonstrates that scaling is necessary.**

End of VESM v1.1 — Standing methodology update.